

Aesthetic-Driven Leader-Line Generation for Label Placement

Anonymous CVPR submission

Paper ID 12655

Abstract

Leader-lines are widely used to connect text annotations and their target elements in various applications. Their arrangement quality strongly affects user attention and cognitive load. The widely used direct leader-lines are easy to implement, but often lack visual appeal. In addition, prior solutions, constrained by predefined templates, struggle to achieve crossing-free arrangements in more complex scenes. To address these challenges, we formulate leader-line generation as a sequential decision problem, where rich aesthetic rules are represented as cost functions. With this novel formulation, we design a leader-line agent (LeLA) to learn effective arrangements. The agent generates diverse candidate paths for each leader-line request. It chooses the valid path by observing the local request features and global arrangement features. To balance the diversity and quality of the leader-line arrangement, we introduce a training-free refinement stage to unify the leader-line styles. Through comprehensive experiments, our LeLA demonstrates its superiority in leader-line generation compared with popular baselines.

1. Introduction

Text annotations, commonly referred to as labels, are widely used to convey supplementary information about scene elements across graphic design [12, 17] and information visualization [15, 44]. Labels are typically connected to their targets via leader-lines, which guide user attention and reduce cognitive load [5, 39]. Accordingly, leader-line arrangement quality directly affects information-transfer efficiency and user experience.

Despite the importance of leader-lines, progress in their generation has lagged far behind that of label placement. First, straight leader-lines often lack aesthetic appeal. Ranging from annotating geographic maps with city names [4] to locating objects in virtual environments with visualization guidance [41], naive straight-line label-target connections are common [8, 9, 12, 44]. Although simple to implement, they often cause crossings and occlusions, harming

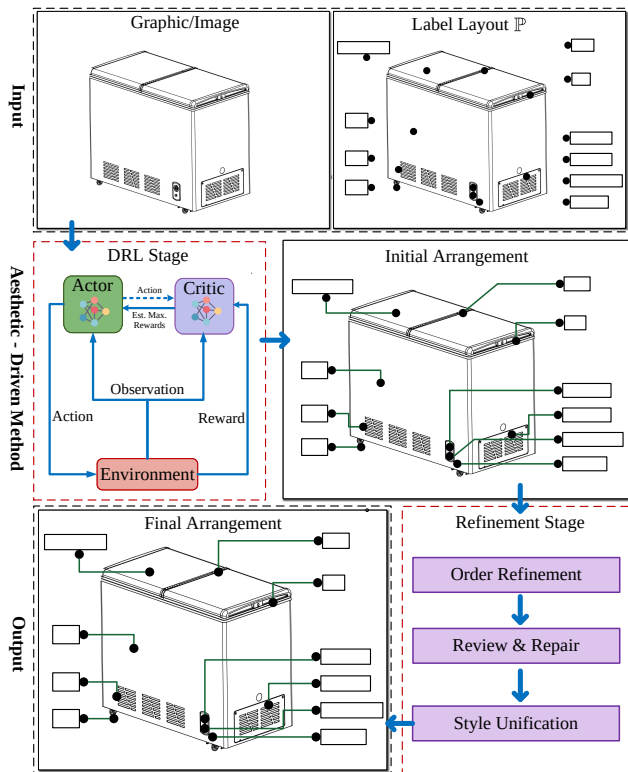


Figure 1. Given the graphic and label layout, the Leader-Line Agent predicts an initial leader-line arrangement, which is then polished by the refinement stage to produce the final arrangement.

visual quality and information-transfer efficiency. Second, prior work is scarce and rigid, with limited practical flexibility. Some studies addressing the boundary labeling problem have designed exact algorithms that seek crossing-free leader-line arrangements by minimizing total path length or the number of bends [3, 17]. However, because these methods are tailored to predefined templates, their leader-line aesthetics are unsatisfactory in more complex scenes.

In this work, we address both challenges with novel contributions. First, we propose a deep reinforcement learning (DRL) solution for leader-line generation (Fig. 1). We formulate the task as a Markov Decision Process (MDP) [36], treating each sample as an ordered sequence of requests.

Within an actor-critic framework, we design a *Leader-Line Agent* (LeLA) to produce leader-lines. The agent provides multiple stylistic candidate paths for each request and adaptively adjusts bend points and segment angles. Its actor selects a valid path using the request’s local cues and the arrangement’s global context. This flexibility allows leader-lines to fit the specific scene dynamically rather than being constrained by rigid, predefined templates. In addition, the agent supports the incorporation of rich aesthetic rules as cost functions to ensure visual quality.

Second, to balance arrangement aesthetics while supporting multiple styles, we introduce a training-free refinement stage. Given the agent’s leader-line arrangement, the stage swaps generation orders whenever the aesthetic cost decreases and re-plans paths that violate hard constraints (line-line intersections and content obstructions). In particular, we design a style-unification algorithm that determines the arrangement’s dominant style and re-plans non-compliant paths to match it. This post-processing step reliably removes residual violations and harmonizes style without retraining.

In summary, our main contributions include:

- We formulate leader-line generation as a sequential decision problem. This novel formulation allows not only multiple stylistic candidate paths for each leader-line, but also more flexible aesthetic rules.
- We develop a novel Leader-Line Agent, which we believe is the first DRL-based solution for leader-line generation. Its actor uses local request features and global arrangement features to determine a valid path for each request.
- We present a training-free refinement stage, which clears residual violations and unifies style for the agent’s leader-line arrangement.
- In carefully designed experiments, LeLA outperforms alternatives in quantitative and qualitative evaluations.

2. Related Work

Leader-Line Generation. The methodologies for label placement have evolved significantly over the years. Early studies focus on geometry- and saliency-based optimization methods, aiming to enhance label stability and legibility [15, 19, 37, 38, 45]. With advances in deep and reinforcement learning, recent work has introduced more data-driven and adaptive placement techniques [4, 7, 14, 16, 24, 30, 42, 43]. Despite substantial advances in label placement, leader-lines, the visual bridge between labels and their targets, have lagged behind in generation and aesthetics. Straight leader-lines directly connecting labels to their targets are the most common approach. While simple to implement, this naive approach is prone to leader-line crossings and occlusions of critical elements. To ensure visual quality and information-transfer accuracy, some methods pursue crossing-free leader-line arrangements. Bekos

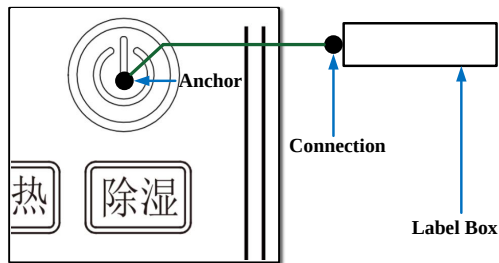


Figure 2. Components of a leader-line request: the anchor on the target element, the label box containing the text, and the connection where the leader-line attaches.

et al. [3] formulate leader-line arrangement as an assignment problem in graph theory, minimizing the total length of octilinear leader-lines. Kindermann *et al.* [17] address the multi-sided boundary labeling problem, proposing a dynamic-programming-based algorithm with axis-parallel leader-lines containing at most one bend. Bastert and Fekete [2] prove that even the simplified Path Selection and Embedding problem, which minimizes either the number of bends or the total leader-line length on a matching graph, is NP-hard. Because of this complexity, exact algorithms [3, 17] scale poorly to complex cases with larger problem sizes and higher aesthetic requirements.

Reinforcement Learning. Reinforcement learning (RL) has emerged as a powerful computational framework for solving sequential decision-making problems, enabling agents to learn effective policies through continual interaction with the environment [20, 28, 40]. The rise of DRL traces back to the Deep Q-Network [29], which introduced convolutional neural networks for end-to-end policy learning. Building on this foundation, Schulman *et al.* proposed Proximal Policy Optimization (PPO) within the actor-critic paradigm, introducing a clipped surrogate objective that effectively stabilizes policy-gradient training [34]. In recent years, RL has achieved remarkable successes across large language models [6, 23], embodied intelligence [27, 31], and autonomous driving [10, 11]. Representative milestones span the AlphaGo series’ breakthroughs in complex strategic reasoning [35] and the emergence of reasoning-oriented generative AI exemplified by DeepSeek-R1 [13].

Our work addresses the challenges in leader-line arrangement by modeling it as a sequence of local decision problem. Our method is the first to use RL for this task, and shows promising performance compared with previous arts.

3. Problem Formulation

In this paper, we deal with the scenarios where labels are placed in a graphic/image and connected to their target elements via leader-lines. Labels are typically represented as

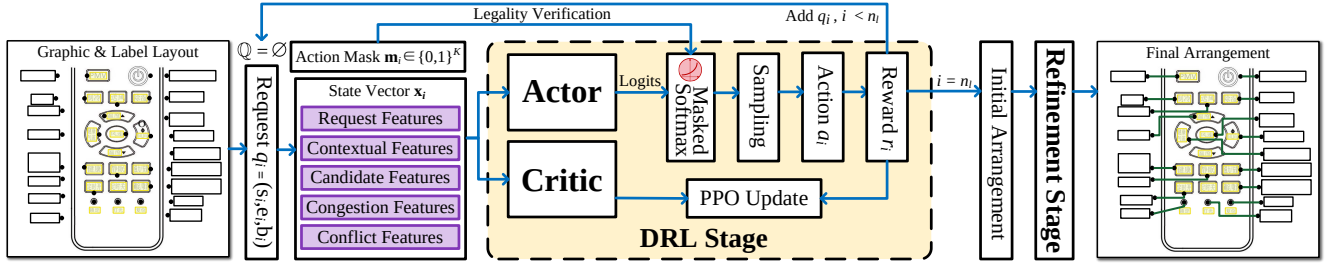


Figure 3. Architecture of LeLA. First, the leader-line requests are formed based on the input graphic and label layout. Next, for each request, LeLA receives an observation and selects an action from a set of candidate paths, guided by a valid action mask. This action yields a reward and adds a leader-line to the initial arrangement. LeLA is implemented as an actor-critic agent and trained with PPO. After all requests are processed, the arrangement is further optimized by the refinement stage.

rectangular boxes. As illustrated in Fig. 2, the target-side endpoint is the anchor, and the label-side endpoint is the connection. In this setting, given a set of labels and associated anchors and image, we define the leader-line generation problem as follows:

- **Input:** A graphic/image of size $H \times W$, and a label layout $\mathbb{P} = \{(s_i \in \mathbb{R}^2, e_i \in \mathbb{R}^2, b_i \in \mathbb{R}^4)\}_{i=1}^{n_l}$, where s_i and e_i are the anchor and label connection point of the i -th label, respectively. Each label box b_i is specified by the coordinates of its top-left and bottom-right corners. n_l is the number of labels/leader-lines.
- **Output:** Leader-lines $\mathbb{L} = \{l_i\}_{i=1}^{n_l}$ that are collectively collision-free and aesthetically pleasing, where each l_i is a polyline (*i.e.*, a sequence of connected line segments) connecting the anchor s_i to the connection e_i .

We formulate leader-line generation as a finite-horizon MDP, treating an input sample as an ordered sequence of requests $\mathbb{Q} = \{q_i\}_{i=1}^{n_l}$. Each leader-line request $q_i = (s_i, e_i, b_i)$ contains an associated anchor s_i , connection e_i , and label box b_i . For each request, LeLA receives an observation o_i , executes an action a_i , and obtains a reward r_i , progressively constructing the leader-line arrangement:

- **Observation o_i :** $o_i = (x_i, m_i)$ consists of a state vector x_i and an action mask m_i . The state vector is constructed from the current request q_i , the already placed leader-lines $\mathbb{L}' = \{l_j\}_{j=1}^{i-1}$, and the pending requests $\mathbb{Q}' = \{q_k\}_{k=i+1}^{n_l}$ to be processed. K candidate paths are dynamically generated for q_i . The binary action mask $m_i \in \{0, 1\}^K$ indicates which of these paths are valid.
- **Action a_i :** The action space of LeLA is a discrete set of valid candidate paths. The action is predicted from the current observation o_i , and denoted by an index $a_i \in \{0, \dots, K-1\}$ corresponding to a valid path. Therefore, the policy $\pi(a_i | o_i)$ learned by LeLA is constrained by the action mask m_i , ensuring that only valid actions can be selected.
- **Reward r_i :** A signal reflecting the immediate and future impact of the action a_i on an aesthetic cost.

Our goal is to train the policy $\pi(a_i | o_i)$ that maximizes

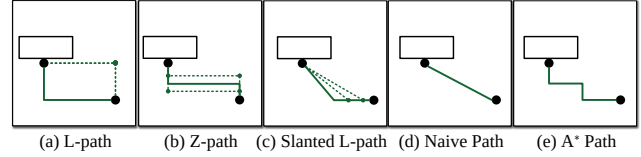


Figure 4. Five types of candidate paths, where the dashed lines indicate multiple candidate paths of one type.

the expected cumulative reward.

4. Methodology

The proposed LeLA, as shown in Fig. 3, consists of two main stages: arrangement generation and refinement.

- **Arrangement Generation.** LeLA first generates candidate paths for the current request. It then encodes the arrangement context into a state vector, which includes the request, the already placed leader-lines, and the pending requests. After computing the validity mask for the candidates, the state vector is passed to the actor-critic network, which selects one valid path for the current request. The environment transits to the next state and returns a reward to guide learning. Repeating this process for all requests produces the initial leader-line arrangement.
- **Arrangement Refinement.** This stage applies deterministic post-processing optimizations to the initial arrangement, including swapping leader-line generation orders, replanning paths, and harmonizing the arrangement style. The final crossing-free arrangement is then produced.

4.1. DRL-based Arrangement Generation

Candidate Path Generation. The action space is a discrete set of candidate paths. For each request $q_i \in \mathbb{Q}$, K candidate paths are dynamically generated, derived from five path types (Fig. 4), with priorities assigned from high to low:

- **Bend Paths (L-path, Z-path, Slanted L-path).** An L-path consists of a horizontal and a vertical segment, forming one bend. The order of the segments is variable. In addition, an L-path undergoes a label box overlap check. If the

connection-side segment is parallel and too close to a label box boundary, they will visually overlap and hurt aesthetics. Therefore, a segment will be slightly tilted to ensure it stays away from the edge. The Z-path is more complex, consisting of one horizontal segment and two vertical segments, forming two bends. The horizontal segment can slide along the two vertical segments to different positions, resulting in multiple distinct paths. The slanted L-path evolves from the orthogonal L-path, with the bend point moving along the horizontal or vertical segment in fixed steps, determined by a specific ratio. As a result, multiple one-bend slanted L-paths are generated. We use only paths with right or obtuse bend angles.

- *Naive Path.* A naive path has a lower priority than the above three paths and is generated only when the anchor and the connection are aligned along the same axis.
- *Constrained A* Path.* If all the above paths fail, LeLA will adopt a constrained A* search algorithm. It can only search for paths along an orthogonal grid, and any path containing more than three bends will be discarded.

Environment Design. For the request q_i , LeLA observes the state vector $\mathbf{x}_i = [\mathbf{x}_i^r, \mathbf{x}_i^l, \mathbf{x}_i^c, \mathbf{x}_i^m, \mathbf{x}_i^s]$, described as:

- *Request feature* $\mathbf{x}_i^r = [\mathbf{s}_i, \mathbf{e}_i, \mathbf{c}_i] \in \mathbb{R}^6$. It contains the 2D coordinates of the request's anchor $\mathbf{s}_i$, connection $\mathbf{e}_i$, and label center $\mathbf{c}_i$ (computed from $\mathbf{b}_i$). The anchor and connection clearly define the start and end points of the current leader-line. The label center helps LeLA capture the relative position of the connection within the label box.
- *Contextual feature* $\mathbf{x}_i^l \in \mathbb{R}^3$. Statistics about the leader-lines already generated, $\mathbb{L}'$, including their count, average length, and total length.
- *Candidate feature* $\mathbf{x}_i^c \in \mathbb{R}^{15K}$. Geometric features of each candidate path are extracted and then concatenated into a single feature vector $\mathbf{x}_i^c$. These features include: path statistics (length and bend count), a one-hot vector for path types, a histogram of bend angles ($0^\circ/45^\circ/90^\circ/135^\circ$ bends), the distance from the anchor to the connection, the minimum distance from the path to $\mathbb{L}'$, and a score measuring the straightness of the connection-side segment in the horizontal and vertical directions. The score is 1.0 for horizontal or vertical segments, 0.0 for 45° angles, and linearly interpolated between 0.0 and 1.0 for other angles. Zero-padding is applied when there are less than K candidate paths.
- *Congestion feature* $\mathbf{x}_i^m \in \mathbb{R}^{2hw}$. We resize the given image to $h \times w$ and construct a grid of $h \times w \times 2$, where the two channels encode the spatial occupancy of generated leader-lines $\mathbb{L}'$ and pending requests $\mathbb{Q}'$. The value of each cell represents congestion, with higher values for cells crossed by more paths.
- *Conflict feature* $\mathbf{x}_i^s \in \mathbb{R}^3$. For current request q_i and each pending request $q_j \in \mathbb{Q}'$, assuming both use a naive path,

their conflict score c^{ij} is computed as:

$$c^{ij} = c_1^{ij} + \frac{2}{1 + c_2^{ij}} + c_3^{ij} \quad (1)$$

where c_1^{ij} measures the intersection of the paths, c_2^{ij} is the shortest distance between them, and c_3^{ij} is the cosine of the angle between them. Finally, $\mathbf{x}_i^s$ collects the max, mean, and std of the conflict scores between q_i and all pending requests, thus quantifies the potential future conflicts caused by the current request.

LeLA performs the action a_i based on the state vector $\mathbf{x}_i$, selecting a path from the candidate paths. We calculate the binary action mask $\mathbf{m}_i$ to identify which paths for the request q_i are valid. A path is considered valid if it does not intersect with any already placed leader-lines $\mathbb{L}'$. In case of no valid candidates, the first candidate is marked as valid to ensure LeLA can always proceed.

To guide LeLA, we design the reward function to estimate the reward r_i after LeLA executes the action a_i . First, we propose the aesthetic cost function $\mathcal{C}$. It is the weighted sum of penalty terms for undesirable arrangement cases. The aim is to find the arrangement $\mathbb{L}^*$ to minimize the cost:

$$\mathbb{L}^* = \arg \min_{\mathbb{L}} \mathcal{C}(\mathbb{L}), \quad \mathcal{C}(\mathbb{L}) = \sum_{k=1}^{n_p} w_k \cdot \mathcal{P}_k(\mathbb{L}) \quad (2)$$

where n_p denotes the number of penalty terms. We categorize the penalties into three types based on their design and impact on leader-line arrangement quality. More details are provided in the supplementary material:

- *Hard constraints.* Cases that severely impact the readability of the leader-line arrangement. These are assigned the highest weights and must be avoided. An example is the case when a leader-line intersects another.
- *Major violations.* Significant aesthetic violations that are discouraged and assigned high weights to minimize. An example is when a line segment runs parallel and too close to a label box edge.
- *Minor violations.* Used for fine-tuning the arrangement. These are not essential for readability but help improve visual quality. For example, a small penalty for total line length encourages simpler, more compact lines.

The reward function consists of the cost drop $\Delta\mathcal{C}$ and the look-ahead penalty $\mathcal{P}_l$. The cost drop is the positive change in aesthetic cost, $\Delta\mathcal{C}_i = \mathcal{C}(\mathbb{L}_{i-1}) - \mathcal{C}(\mathbb{L}_i)$, where $\mathbb{L}_{i-1}$ and $\mathbb{L}_i$ are the leader-line arrangements at the previous and current steps. The look-ahead penalty estimates the future cost of the current action a_i on the pending requests $\mathbb{Q}'$, and is computed using the congestion and conflict features. The reward function is computed as:

$$r_i = \Delta\mathcal{C}_i - \lambda_i \mathcal{P}_l(a_i) \quad (3)$$

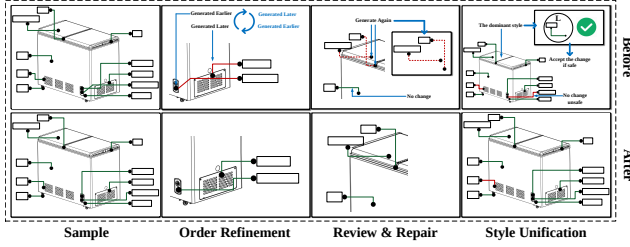


Figure 5. Demonstration of the three sequential optimizations in the refinement stage, namely order refinement, review & repair, and style unification.

where λ_i is a weight that gradually increases during training. This allows LeLA to focus on optimizing the immediate cost early in training and gradually shift to penalizing future conflicts as it improves.

Agent Architecture. LeLA employs an actor-critic architecture, as depicted in Fig. 3. At each decision step, the observation o_i is passed to the actor and critic networks: (1) the critic outputs a scalar estimating the expected cumulative reward from the current state, and (2) the actor outputs a vector of logits over the K candidate actions. These logits are combined with the binary action mask $\mathbf{m}_i$, which sets the probability of any invalid candidate to zero. Finally, LeLA selects the discrete action a_i by sampling from the resulting masked categorical distribution.

4.2. Arrangement Refinement

To effectively balance the diversity and aesthetics of the leader-line arrangement, we introduce the refinement stage, as illustrated in Fig. 5. First, we adjust the generation order of leader-lines. We scan the initial arrangement and check adjacent pairs. If swapping their order reduces the aesthetic cost, we adopt the new order and update the cost. Next, we fix key violations, including line-line intersections and content occlusions. We detect leader-lines that intersect with either other leader-lines or regions with important semantic information. In addition to labels, we consider content areas, *i.e.*, regions containing the main information of the image. For each violating leader-line, we regenerate its path.

Style Unification. Finally, we design a style-unification algorithm (shown in Alg. 1) to improve visual coherence. The algorithm first identifies the dominant style by counting the frequency of each leader-line type, selecting the most frequent one as the dominant style. We consider only L-path, Z-path, and Slanted L-path. The algorithm then regenerates any leader-lines that do not match the dominant style (*e.g.*, a Z-path line in an L-path dominant layout), ensuring that no *Hard Constraints* are violated. Completely straight leader-lines are left unchanged, as they are considered aesthetically pleasing. This process produces the final leader-line arrangement.

Algorithm 1 Style Unification Algorithm

```

1: procedure UNIFYSTYLE( $\mathbb{L}_{init}, \mathbb{Q}$ )
2:    $\mathbb{L} \leftarrow \mathbb{L}_{init}$ 
3:    $S_{compete} \leftarrow \{\text{'l\_path'}, \text{'z\_path'}, \text{'slanted.l\_path'}\}$ 
4:    $S_{dom} \leftarrow \text{FindDominantStyle}(\mathbb{L}, S_{compete})$ 
5:   if  $S_{dom} = \text{null}$  then
6:     return  $\mathbb{L}$  ▷ No dominant style
7:   end if
8:    $S_{ignore} \leftarrow \{\text{'axis\_aligned\_straight'}, \text{'astar\_path'}\}$ 
9:   for  $i \leftarrow 1$  to  $\text{Length}(\mathbb{L})$  do
10:     $S_{curr} \leftarrow \text{ClassifyPath}(\mathbb{L}[i])$ 
11:    if  $S_{curr} = S_{dom}$  or  $S_{curr} \in S_{ignore}$  then
12:      continue ▷ Skip dominant or ignored
13:    end if
14:     $p_{new} \leftarrow \text{FindSafeReplacement}(\mathbb{L}[i], S_{dom}, \mathbb{L}, \mathbb{Q})$ 
15:    if  $p_{new} \neq \text{null}$  then
16:       $\mathbb{L}[i].\text{path} \leftarrow p_{new}$  ▷ Apply new path
17:    end if
18:  end for
19:  return  $\mathbb{L}$ 
20: end procedure

```

5. Experiments

To evaluate the performance of LeLA, we conduct comprehensive experiments: (1) a comparison with baselines using quantitative and qualitative analyses, (2) an ablation study to assess the key components, and (3) a user study to evaluate the perceptual quality and user preferences for the generated arrangements. The implementation details of LeLA are provided in the supplementary material. We will release our algorithm upon the publication of this paper.

5.1. Experimental Setup

Dataset. Our experiments are conducted on the SWU-AMIL dataset [30]. The dataset contains 869 professionally designed label arrangements collected from commercial product illustrations, spanning 11 appliance categories. We follow the official 695/174 split for training and testing, respectively. The dataset exhibits significant diversity; in the test set, samples contain an average of 9.3 leader-lines, with the counts ranging from 1 to 25.

Baselines. We compare our method against several baselines: (1) a naive method that draws direct leader-lines and is commonly used as a basic benchmark [1, 21, 22, 25, 26], and (2) two representative traditional methods, namely the octilinear leader method (OL) [3] and the multi-sided boundary labeling method (MBL) [17]. Since there is no official code available, we reproduce the two methods and tune their parameters to optimize the results. The performance of all methods is evaluated against the ground truth arrangements created by professional designers.

Table 1. Comparison of leader-line arrangement quality (AAC, AI, and OR) and efficiency on the SWU-AMIL dataset. “Time” denotes the average time (in seconds) to generate one complete arrangement, calculated over all test samples. For all metrics, lower is better. Numbers in bold indicate the best performance.

Method	AAC	AI	OR(%)	Time(s)
GT	2.4e5	0.027	12.64	N/A
Naive	5.4e5	0.016	24.71	0.000
OL [3]	4.2e4	0.005	23.56	0.012
MBL [17]	1.8e5	0.069	17.82	0.000
LeLA	1.8e4	0.008	0.57	20.966

Evaluation Metrics. To conduct a comprehensive evaluation of leader-line generation, we introduce three metrics: 1) Average Arrangement Cost (AAC) is computed using the aesthetic cost function $\mathcal{C}$ (Eq. 2). It provides a method-agnostic evaluation by assessing the final geometry of any arrangement, regardless of how it was generated. 2) Average Intersections (AI) measures visual clarity, representing the average number of crossings between all non-adjacent leader-line segments in an arrangement. 3) Obstruction Rate (OR) measures functional usability as the percentage of generated arrangements with at least one leader-line intersecting a label box or content box.

5.2. Evaluation Results

Quantitative Analysis. Table 1 presents the quantitative comparison between LeLA and the baselines. LeLA achieves strong performance across all arrangement quality metrics. Specifically, compared to the traditional method OL, which achieves the best performance on AI, LeLA reduces AAC by 57.1%. More importantly, it decreases OR from 23.56% to 0.57%, effectively eliminating collisions with key content, while its AI (0.009) remains relatively small. These results demonstrate that LeLA can handle complex spatial dependencies and aesthetic constraints that rule-based approaches have difficulty modeling. Moreover, Tab. 1 shows a classic quality–efficiency trade-off. While traditional methods are faster because of their simplified heuristics, this speed comes at a significant cost to arrangement quality. In contrast, LeLA takes about 21 seconds on average to generate an arrangement and is nearly as fast as the baselines for most samples, while delivering higher quality. We therefore position our method not as a real-time rendering tool, but as a helpful “co-pilot” for design workflows. In such scenarios, spending a few seconds of computation to obtain a well-designed leader-line arrangement that reduces manual adjustment is highly valuable.

Qualitative Analysis. Figure 6 shows arrangements generated by different methods for visual comparison. LeLA produces more coherent and aesthetically pleasing arrangements: leader-lines are well organized, navigate around

Table 2. Ablation studies of the core components of LeLA.

Method	AAC	AI	OR(%)	Time(s)
LeLA-NoRefine	5.3e4	0.027	2.87	3.830
LeLA-NoUnify	5.3e4	0.027	2.87	3.857
LeLA-NoHard	5.4e4	0.027	4.02	4.238
LeLA-NoMajor	4.0e5	0.019	2.87	3.333
LeLA-NoMinor	7.9e4	0.035	1.72	4.182
LeLA-L	8.3e4	0.038	9.20	1.760
LeLA-Z	8.2e4	0.036	5.75	3.147
LeLA-Slanted L	5.4e5	0.016	20.69	1.322
LeLA-Naive	5.4e5	0.016	20.69	1.183
LeLA	1.8e4	0.008	0.57	20.966

content areas, and reduce clutter, making the correspondence between labels and targets clear. In contrast, the baselines yield lower-quality layouts. OL and MBL often generate leader-lines that cut across important visual elements, compromising usability, while the naive approach finds shorter paths at the cost of many crossings and occlusions. Overall, the qualitative results indicate that LeLA produces arrangements that better align with human perceptual principles of clarity and good design.

5.3. Ablation Study

We conduct a comprehensive ablation study to assess the contributions of our framework’s key components. As shown in Tab. 2, we compare our model (LeLA, majorly LeLA-Refine where refinement stage are ablated) against variants where specific components are disabled.

Refinement Stage. Removing the refinement stage (LeLA-NoRefine) shows that AAC increases by 194.4% (from 1.8e4 to 5.3e4) and AI rises from 0.008 to 0.027, indicating that the DRL agent depends on this stage to clean up residual violations. Disabling only the style unification step (LeLA-NoUnify) yields a similar AAC (5.3e4) and AI (0.027); this numerical similarity to LeLA-NoRefine suggests the DRL agent produces high-quality initial layouts that rarely trigger the order and repair steps. While these steps remain effective for the few violations that occur, this result clearly shows that style unification is essential for achieving the final low-cost, high-quality arrangement.

Aesthetic Cost Components. We further analyze the cost function by ablating the penalty groups from the LeLA-NoRefine baseline. Removing the penalties for Hard Constraints (LeLA-NoHard) yields a similar AAC (5.4e4), while removing Minor Violations (LeLA-NoMinor) increases AAC to 7.9e4 (a 49.1% increase). Removing the Major Violations group (LeLA-NoMajor) has the largest impact, pushing AAC up to 4.0e5, which is over 650% higher than the LeLA-NoRefine baseline. These results show that the agent’s policy relies on these penalties to produce acceptable initial layouts, even before refinement.

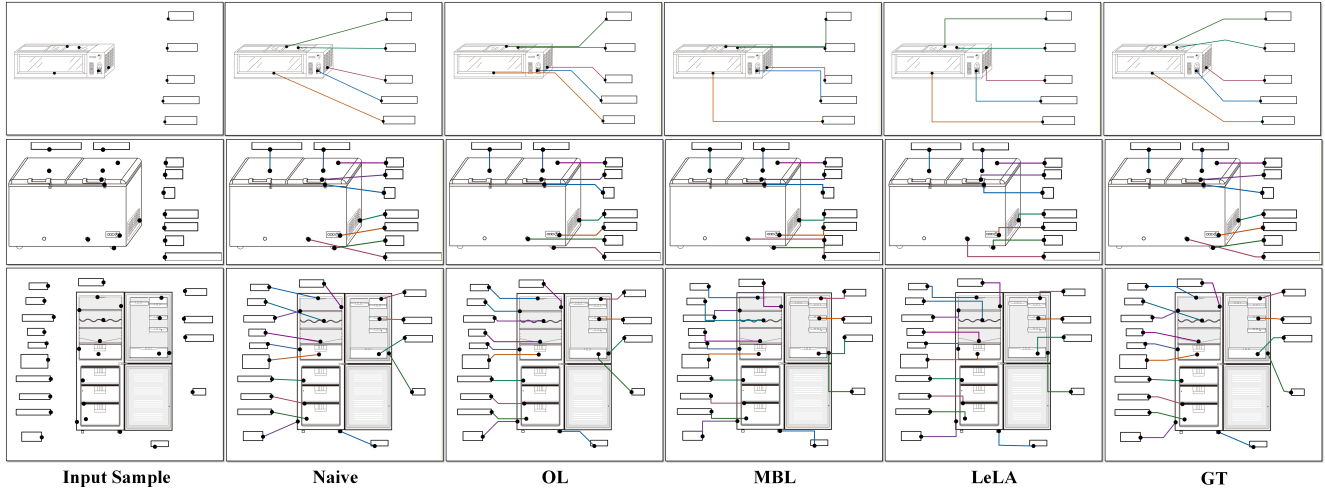


Figure 6. Visualization results on the SWU-AMIL dataset with varying leader-line densities and anchor distributions.

Candidate Paths. Finally, we study the effect of restricting the agent to a single path type, again relative to the LeLA-NoRefine baseline. Using only Z-path (LeLA-Z) or only L-path (LeLA-L) increases AAC to $8.2e4$ and $8.3e4$, respectively (a 55–57% increase). Restricting the agent to only slanted L-path (LeLA-Slanted L) or only naive path (LeLA-Naive) is even worse, with AAC rising to $5.4e5$. These results show that the agent needs a diverse set of candidate styles to find good arrangements.

5.4. User Study

To assess the perceptual quality of our generated arrangements, we conducted a user study. The study was designed to quantify the impact of different leader-line methods on user performance in a set of visual-cognitive tasks.

5.4.1. Settings

Participants and Setup. We recruited 26 participants (14 male, 12 female; age 17-34), all with normal or corrected-to-normal vision. The experiment was administered via a custom web-based questionnaire that participants completed on their personal computers. Before the trials, the system prompted them to adjust their browser zoom level to obtain a clear view of the test images and answer fields.

Methods, Samples, and Tasks. We evaluated three methods: MBL, ground-truth arrangements (GT), and LeLA. All samples were taken from the SWU-AMIL test set. We designed three visual tasks on leader-lines, ranging from local information retrieval to global comprehension: 1) Matching. Participants located the label corresponding to a highlighted red anchor and entered the numeric index on that label. This task measured the legibility of the target-label association provided by the leader-line. 2) Calculating. Two anchors were highlighted simultaneously. Participants found their respective labels and entered the absolute differ-

ence between the two numeric indices. This task assessed the clarity of local spatial relationships among leader-lines. 3) Scanning. Participants scanned the entire arrangement and counted the total number of bends across all leader-lines. This task evaluated the overall visual complexity and structural clarity of the arrangement.

Study Design. The experiment employed a 3 (Method) $\times$ 3 (Task) $\times$ 2 (Complexity) full-factorial, within-subjects design. Based on a statistical analysis of the test set, we defined complexity levels using the 75th percentile of the leader-line counts as the threshold: low-density (1-13 lines) and high-density (14-25 lines). Each participant completed $3 \times 3 \times 2 \times 3$ (Samples) = 54 trials in total.

To mitigate learning and fatigue effects, the three leader-line arrangements for the same sample were never shown consecutively, and low- and high-density arrangements were presented alternatively. Participants provided their demographic information on the login screen. In each trial, they viewed an image of a leader-line arrangement and submitted their response in an input field. They then rated the task’s perceived difficulty on a 7-point Likert scale (1 = very easy, 7 = very difficult) as an indicator of mental workload.

We measured three dependent variables: 1) **accuracy**: correctness of the response (1 = correct, 0 = incorrect); 2) **completion time**: seconds from trial start to response submission, logged automatically; and 3) **perceived difficulty**: subjective rating of mental workload per task.

5.4.2. Results

We conducted a statistical analysis of the data to examine whether there were significant differences among the methods on the three measures. Because a Shapiro–Wilk test indicated that the data were not normally distributed, we used the nonparametric Friedman test for overall comparisons, followed by post hoc tests for pairwise comparisons.

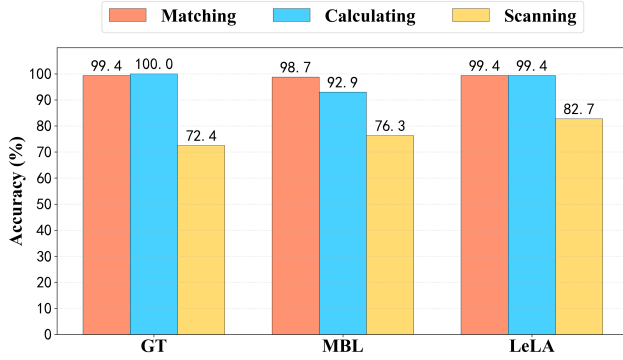


Figure 7. User study results: mean accuracy (%).

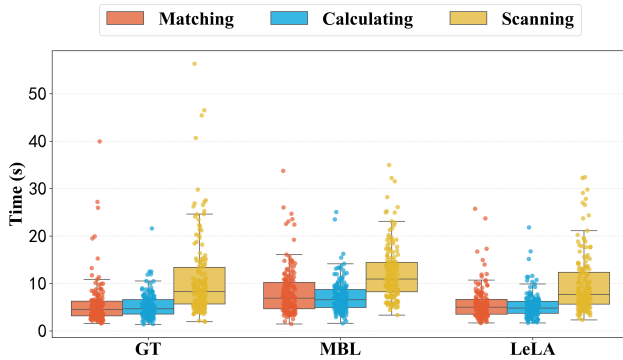


Figure 8. User study results: completion time (s).

Accuracy. Figure 7 shows the mean accuracy for each method. The Friedman test found no significant differences among the methods for the matching task, where performance was near perfect (LeLA: 99.4%; GT: 99.4%). In contrast, significant overall differences were observed for the calculating ($p < 0.001$) and scanning ($p = 0.008 < 0.01$) tasks. For the calculating task, post hoc tests did not reveal any statistically significant pairwise differences. Notably, in the more demanding scanning task, LeLA achieved the highest accuracy (82.7%), which was significantly higher than that of GT (72.4%; $p = 0.016 < 0.05$).

Completion Time. Figure 8 presents the completion times. The Friedman test revealed significant differences among the methods across all three tasks ($p < 0.002$). For both the matching and calculating tasks, post hoc tests showed that completion times were significantly shorter for LeLA than for MBL ($p < 0.001$ for both), and did not differ significantly from GT. Crucially, for the scanning task, LeLA was significantly faster than MBL ($p = 0.001$).

Perceived Difficulty. Figure 9 shows subjective ratings of perceived difficulty. The Friedman test found significant differences across all tasks ($p < 0.001$). Post hoc analysis indicated that MBL yielded significantly higher perceived difficulty than both LeLA and GT across all tasks ($p < 0.001$). For the matching and calculating tasks, LeLA

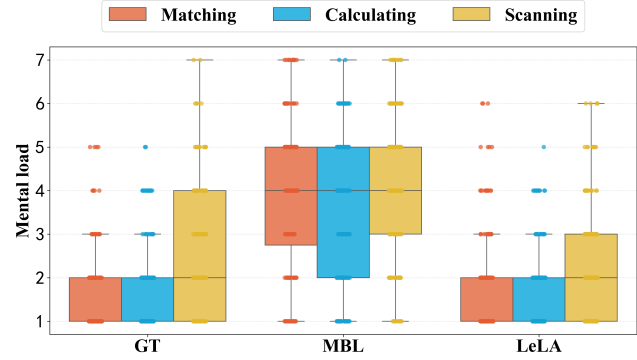


Figure 9. User study results: participants’ subjective ratings of perceived difficulty (1-7 Likert scale).

was rated significantly lower than MBL ($p < 0.001$ for both) and did not differ significantly from GT. For the scanning task, LeLA was again rated significantly less demanding than MBL ($p < 0.001$) and did not differ significantly from GT. This suggests that LeLA leads to a cognitive load comparable to that of human designers.

Summary. The user study results show that LeLA improves user performance in visual-cognitive tasks. In terms of accuracy, it is robust and significantly outperforms GT in the complex scanning task. For completion time, it is significantly faster than MBL across all tasks, while achieving efficiency comparable to GT on simpler tasks. Ratings of perceived difficulty indicate that the cognitive burden of using LeLA is comparable to or lower than that of GT, and substantially lower than that of MBL. Taken together, these findings show that our DRL-based approach not only produces quantitatively and aesthetically superior arrangements but also translates these gains into practical benefits in user performance and experience.

6. Conclusion and Discussion

In this paper, we model leader-line generation as a Markov Decision Process. Based on this formulation, we develop a Leader-Line Agent that generates diverse candidate paths and selects a valid one using local and global arrangement features. A refinement stage further reduces aesthetic violations. Our method, as the first attempt using RL for the task, produces clear better performance than previous arts.

Limitations. Despite encouraging results achieved, our method has limitations. First, it currently does not handle correspondences between a single label and multiple anchors, or between multiple labels and a single anchor. Second, the cost function contains relatively many penalty terms. In future work, we plan to adjust the environment design to address the above issues, and to study the relationships among aesthetic rules for simplification.

References

- [1] Ronald Azuma and Chris Furmanski. Evaluating Label Placement for Augmented Reality View Management. In *Proc. of ISMAR*, pages 66–75, 2003. 5
- [2] Oliver Bastert and Sándor P. Fekete. *Geometrische Verdrahtungsprobleme*. Universität zu Köln, Köln, Germany, 1996. Dissertation. 2
- [3] Michael A Bekos, Michael Kaufmann, Martin Nöllenburg, and Antonios Symvonis. Boundary Labeling with Octilinear Leaders. *Algorithmica*, 57(3):436–461, 2010. 1, 2, 5, 6
- [4] Petr Bobák, Ladislav Čmolík, and Martin Čadík. Reinforced Labels: Multi-Agent Deep Reinforcement Learning for Point-Feature Label Placement. *IEEE Trans. Vis. Comput. Graph.*, 30(9):5908–5922, 2024. 1, 2
- [5] Zahra Borhani, Prashast Sharma, and Francisco R. Ortega. Survey of Annotations in Extended Reality Systems. *IEEE Trans. Vis. Comput. Graph.*, 30(8):5074–5096, 2024. 1
- [6] Yuji Cao, Huan Zhao, Yuheng Cheng, Ting Shu, Yue Chen, Guolong Liu, Gaoqi Liang, et al. Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods. *IEEE Trans. Neural Netw. Learn. Syst.*, 36(6):9737–9757, 2025. 2
- [7] Zhu-Tian Chen, Daniele Chiappalupi, Tica Lin, Yalong Yang, Johanna Beyer, and Hanspeter Pfister. RL-LABEL: A Deep Reinforcement Learning Approach Intended for AR Label Placement in Dynamic Scenarios. *IEEE Trans. Vis. Comput. Graph.*, 30(1):1347–1357, 2024. 2
- [8] Ladislav Čmolík, Václav Pavlovic, Hsiang-Yun Wu, and Martin Nöllenburg. Mixed Labeling: Integrating Internal and External Labels. *IEEE Trans. Vis. Comput. Graph.*, 28(4):1848–1861, 2022. 1
- [9] Semir Elezovikj, Jianqing Jia, Chiu C. Tan, and Haibin Ling. PartLabeling: A Label Management Framework in 3D Space. *Virtual Reality & Intelligent Hardware*, 5(6):490–508, 2023. 1
- [10] Shuo Feng, Xintao Yan, Haowei Sun, Yiheng Feng, and Henry X. Liu. Intelligent Driving Intelligence Test for Autonomous Vehicles With Naturalistic and Adversarial Environment. *Nature Communications*, 12:748, 2021. 2
- [11] Shuo Feng, Haowei Sun, Xintao Yan, Haojie Zhu, Zhengxia Zou, Shengyin Shen, and Henry X. Liu. Dense Reinforcement Learning for Safety Validation of Autonomous Vehicles. *Nature*, 615:620–627, 2023. 2
- [12] Martin Fink, Jan-Henrik Haunert, André Schulz, Joachim Spoerhase, and Alexander Wolff. Algorithms for Labeling Focus Regions. *IEEE Trans. Vis. Comput. Graph.*, 18(12):2583–2592, 2012. 1
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeek-R1 Incentivizes Reasoning in LLMs Through Reinforcement Learning. *Nature*, 645(8081):633–638, 2025. 2
- [14] Jianing Hao, Manling Yang, Qing Shi, Yuzhe Jiang, Guang Zhang, and Wei Zeng. FinFlier: Automating Graphical Overlays for Financial Visualizations With Knowledge-Grounding Large Language Model. *IEEE Trans. Vis. Comput. Graph.*, 31(9):6353–6369, 2025. 2
- [15] Srinidhi Hegde, Jitender Maurya, Ramya Hebbalaguppe, and Aniruddha Kalkar. SmartOverlays: A Visual Saliency Driven Label Placement for Intelligent Human-Computer Interfaces. In *Proc. of WACV*, pages 1110–1119, 2020. 1, 2
- [16] Daichi Horita, Naoto Inoue, Kotaro Kikuchi, Kota Yamaguchi, and Kiyoharu Aizawa. Retrieval-Augmented Layout Transformer for Content-Aware Layout Generation. In *Proc. of CVPR*, pages 67–76, 2024. 2
- [17] Philipp Kindermann, Benjamin Niedermann, Ignaz Rutter, Marcus Schaefer, André Schulz, and Alexander Wolff. Multi-sided Boundary Labeling. *Algorithmica*, 76(1):225–258, 2016. 1, 2, 5, 6
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *Proc. of ICLR*, 2015. 1
- [19] Thomas Köppel, M. Eduard Gröller, and Hsiang-Yun Wu. Context-Responsive Labeling in Augmented Reality. In *Proc. of PacificVis*, pages 91–100, 2021. 2
- [20] Matthew Landers and Afsaneh Doryab. Deep Reinforcement Learning Verification: A Survey. *ACM Comput. Surv.*, 55(14s):1–31, 2023. 2
- [21] Hyunjin Lee and Woontack Woo. Exploring the Effects of Augmented Reality Notification Type and Placement in AR HMD while Walking. In *Proc. of VR*, pages 519–529, 2023. 5
- [22] Tica Lin, Yalong Yang, Johanna Beyer, and Hanspeter Pfister. Labeling Out-of-View Objects in Immersive Analytics to Support Situated Visual Searching. *IEEE Trans. Vis. Comput. Graph.*, 29(3):1831–1844, 2023. 5
- [23] Keliang Liu, Dingkan Yang, Ziyun Qian, Weijie Yin, Yuchi Wang, Hongsheng Li, Jun Liu, et al. Reinforcement learning meets large language models: A survey of advancements and applications across the llm lifecycle. *CoRR*, abs/2509.16679, 2025. 2
- [24] Feiyu Lu, Mengyu Chen, Hsiang Hsu, Pranav Deshpande, Cheng Yao Wang, and Blair MacIntyre. Adaptive 3D UI Placement in Mixed Reality Using Deep Reinforcement Learning. In *Proc. of CHI*, pages 1–7, 2024. 2
- [25] Feiyu Lu, Mengyu Chen, Hsiang Hsu, Pranav Deshpande, Cheng Yao Wang, and Blair MacIntyre. Adaptive Content Placement in Mixed Reality Through Empirical User Behavioral Patterns. In *Proc. of ISMAR*, pages 199–204, 2024. 5
- [26] Ann McNamara, Katherine Boyd, Joanne George, Weston Jones, Somyung Oh, and Annie Suther. Information Placement in Virtual Reality. In *Proc. of VR*, pages 1765–1769, 2019. 5
- [27] Yuan Meng, Zhibin Bing, Xiaoqing Yao, Kejia Chen, Kai Huang, Yang Gao, Fuchun Sun, and Alois Knoll. Preserving and Combining Knowledge in Robotic Lifelong Reinforcement Learning. *Nat. Mach. Intell.*, 7(2):256–269, 2025. 2
- [28] Stephanie Milani, Nicholay Topin, Manuela Veloso, and Fei Fang. Explainable Reinforcement Learning: A Survey and Comparative Review. *ACM Comput. Surv.*, 56(7):1–36, 2024. 2
- [29] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518(7540):529–533, 2015. 2

- [30] Jingwei Qu, Pingshun Zhang, Enyu Che, Yinan Chen, and Haibin Ling. Graph Transformer for Label Placement. *IEEE Trans. Vis. Comput. Graph.*, 31(1):1257–1267, 2025. 2, 5
- [31] Ilija Radosavovic, Tete Xiao, Bike Zhang, Trevor Darrell, Jitendra Malik, and Koushil Sreenath. Real-world Humanoid Locomotion With Reinforcement Learning. *Science Robotics*, 9(89), 2024. 2
- [32] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *J. Mach. Learn. Res.*, 22(268):1–8, 2021. 1
- [33] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *Proc. of ICLR*, 2016. 1
- [34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *CoRR*, abs/1707.06347, 2017. 2, 1
- [35] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. Mastering the Game of Go Without Human Knowledge. *Nature*, 550(7676):354–359, 2017. 2
- [36] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 1998. 1
- [37] Markus Tatzenberg, Denis Kalkofen, Raphael Grasset, and Dieter Schmalstieg. Hedgehog Labeling: View Management Techniques for External Labels in 3D Space. In *Proc. of VR*, pages 27–32, 2014. 2
- [38] Markus Tatzenberg, Valeria Orso, Denis Kalkofen, Giulio Jacucci, Luciano Gamberini, and Dieter Schmalstieg. Adaptive Information Density for Augmented Reality Displays. In *Proc. of VR*, pages 83–92, 2016. 2
- [39] Tobias Volmer, Christian M. Theobalt, and Philipp Slusallek. Floating Labels: Applying Dynamic Label Placement to Augmented Reality. *IEEE Trans. Vis. Comput. Graph.*, 24(11):2925–2935, 2018. 1
- [40] Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, et al. Deep Reinforcement Learning: A Survey. *IEEE Trans. Neural Netw. Learn. Syst.*, 35(4):5064–5078, 2024. 2
- [41] Xiaoheng Wei, Xuehuai Shi, and Lili Wang. Label guidance based object locating in virtual reality. In *Proc. of ISMAR*, pages 441–449, New York, 2022. IEEE. 1
- [42] Peirong Zhang, Jiaxin Zhang, Jiahuan Cao, Hongliang Li, and Lianwen Jin. Smaller But Better: Unifying Layout Generation with Smaller Large Language Models. *Int. J. Comput. Vis.*, 133(7):3891–3917, 2025. 2
- [43] Pingshun Zhang, Enyu Che, Yinan Chen, Bingyao Huang, Haibin Ling, and Jingwei Qu. Mixture of Cluster-Guided Experts for Retrieval-Augmented Label Placement. *IEEE Trans. Vis. Comput. Graph.*, 2026. 2, 3, 5
- [44] Zijie Zheng, Yu He, Ge Yu, and Xi Xu. 3D Label Layout with Binocular Optimization. In *Proc. of SIGGRAPH*, pages 1–2, 2025. 1
- [45] Zijiang Zhou, Lili Wang, and Voicu Popescu. A Partially-Sorted Concentric Layout for Efficient Label Localization in Augmented Reality. *IEEE Trans. Vis. Comput. Graph.*, 27(11):4087–4096, 2021. 2

Aesthetic-Driven Leader-Line Generation for Label Placement

Supplementary Material

7. Symbol Notations

For clarity, Tab. 3 summarizes the key symbols used in this paper along with their respective meanings.

8. Aesthetic Cost Function Details

The aesthetic cost function $\mathcal{C}$ aggregates multiple weighted penalty terms into a single score representing the overall quality of a leader-line arrangement. Table 4 lists all penalty components, their corresponding weights, and brief descriptions of their roles. These weights were empirically chosen to align with established principles of good design and information clarity.

In addition to label boxes, we define content areas to mark important regions in a given graphic. As illustrated in Fig. 10, the content areas are represented as bounding boxes that enclose key semantic information, such as the main product, logos, or other critical visual elements. These areas are treated as high-priority obstacles. The `intrusion.severity` penalty penalizes any leader-line that crosses into these protected zones, ensuring that important information is not obscured.

9. Additional Experiments

9.1. Implementation Details

We train our agent using Maskable Proximal Policy Optimization (PPO) [34], implemented with PyTorch and Stable-Baselines [32], for a total of 200,000 timesteps. Our actor-critic architecture consists of two independent MLPs: one network for the policy (actor) and another for the value (critic). Each network is composed of two fully-connected layers with hidden size 256 and ReLU activations, followed by a linear output head. Both networks take the state vector as input. Training follows a sample-and-update loop: the current policy collects a rollout of 2,048 environment steps, which are then used for multiple epochs of stochastic gradient ascent to update the networks. PPO optimizes the clipped surrogate objective with Generalized Advantage Estimation (GAE) [33]:

$$\mathcal{L}(\theta) = \mathbb{E}_i \left[\min \left(R_i(\theta) \hat{A}_i, \text{CLIP} \left(R_i(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) \right] \quad (4)$$

where $R_i(\theta) = \frac{\pi_\theta(a_i|o_i)}{\pi_{\theta'}(a_i|o_i)}$ is the probability ratio, $\hat{A}_i$ is the advantage estimate, ϵ is the clipping coefficient, and θ and θ' denote the updated and old parameters of the actor-critic

Table 3. Key symbol notations.

Symbol	Meaning
P, L, Q	sets of label information, leader-lines, requests
s, e, b, c l, x, m	anchor point, connection point, label box, label center leader-line, state vector, action mask
$\mathcal{C}, \mathcal{P}, \mathcal{V}$	cost function, penalty function, value function (critic)
K	number of candidate paths
H, W	image size
π	policy (actor)
q, o, a, r	request, observation, action, reward
c	conflict score
r, l, c, m, s	superscripts of state vector components
$n, w \text{ \& } \lambda$	quantity, weights
h, w	grid size

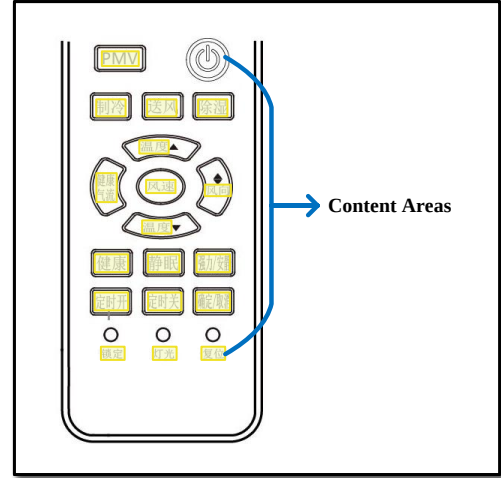


Figure 10. An example of content areas (shown in yellow bounding boxes) defined for a graphic. Our agent is trained to avoid routing leader-lines through them.

networks. The advantage is estimated using GAE:

$$\hat{A}_i = \sum_{l=0}^{\infty} (\gamma \lambda_g)^l \delta_{i+l} \quad (5)$$

where $\delta_i = r_i + \gamma \mathcal{V}(o_{i+1}) - \mathcal{V}(o_i)$ is the temporal-difference residual, $\mathcal{V}(o)$ is the critic’s estimate of the state-value function (the expected cumulative reward from state o), γ is the discount factor, and λ_g is the GAE parameter.

We set the number K of candidate paths for each request to 20. The congestion feature uses a grid of size $h = 36, w = 64$. We use the Adam optimizer [18], with a learning rate initialized to $3e-4$ and decayed linearly

Table 4. Normalized penalty components and weights used in the aesthetic cost function.

Penalty Component	Weight	Role
<i>Hard Constraints</i>		
Intersection	50,000.0	Penalty for each intersection between non-adjacent leader-line segments.
Obstacle	50,000.0	Penalty for each time a leader-line crosses a permanent obstacle (<i>e.g.</i> , a label box).
Excessive_complexity	1,000,000.0	Very high penalty for leader-lines with more than three bends (<i>e.g.</i> , from a complex A* path).
<i>Major Violations</i>		
Straight_diagonal	100,000.0	High penalty for straight lines that are not axis-aligned, discouraging their use unless necessary.
Intrusion_severity	7,500.0	Penalty for leader-lines encroaching on the content areas.
Edge_parallelism	25,000.0	Strong penalty for axis-aligned segments that run too close and parallel to the edges of label boxes.
Monotonicity	5,000.0	Penalty for leader-lines that reverse direction (<i>e.g.</i> , forming a “zig-zag”), which harms readability.
<i>Minor Violations</i>		
Angle_elegance	1,000.0	Penalty applied to sharp, acute angles at bends, encouraging right or obtuse angles.
turn	800.0	General penalty applied for each bend in a leader-line.
Proximity	50.0	Penalty for segments from different leader-lines being too close to each other.
complexity	50.0	Small penalty for leader-lines with more than one bend (<i>e.g.</i> , Z-paths), favoring simpler styles like L-paths.
Non_preferred_angle	30.0	Penalty for line segments not aligned to preferred angles (<i>e.g.</i> , 0°, 45°, 90°).
Alignment	2.0	Minor penalty encouraging bends to align along horizontal or vertical axes.
Parallelism	2.0	Minor penalty encouraging parallelism or orthogonality between different segments in the arrangement.
Length	0.005	Very small penalty proportional to the total length of all leader-lines to favor compactness.

798 over training. All experiments are conducted on a single
 799 NVIDIA GeForce RTX 3090 GPU and an Intel Core i9-
 800 12900K CPU.

801 9.2. Calculation of Evaluation Metrics

802 The three evaluation metrics, AAC, AI, and OR, are defined
 803 as follows.

804 **Average Arrangement Cost (AAC).** This metric is com-
 805 puted by averaging the cost function $\mathcal{C}$ over all test samples.

$$806 \quad \text{AAC} = \frac{1}{N} \sum_{i=1}^N \mathcal{C}(\mathbb{L}_i) \quad (6)$$

807 where N denotes the number of test samples.

Average Intersections (AI). This metric quantifies the
 average number of leader-line crossings per sample.

$$\text{AI} = \frac{1}{N} \sum_{i=1}^N \mathcal{I}(\mathbb{L}_i) \quad (7)$$

where the function $\mathcal{I}$ counts the total number of leader-line
 intersections within a sample.

Obstruction Rate (OR). This metric measures the func-
 tional usability of a leader-line arrangement.

$$\text{OR} = \frac{1}{N} \sum_{i=1}^N \mathcal{O}(\mathbb{L}_i) \quad (8)$$

where the function $\mathcal{O}$ returns 1 if an arrangement contains at
 least one leader-line that intersects any permanent obstacle
 (*e.g.*, label boxes or content areas), and 0 otherwise.

Table 5. Ablation studies of the penalties of the aesthetic cost.

Method	AAC	AI	OR(%)	Time(s)
LeLA-NoRefine	5.3e4	0.027	2.87	3.830
LeLA-Hard	9.7e4	0.037	6.32	3.411
LeLA-Major	8.6e4	0.047	6.90	2.889
LeLA-Minor	4.4e5	0.029	5.75	2.117
LeLA	1.8e4	0.008	0.57	20.966

9.3. Ablation Study

Aesthetic Cost Components. Beyond the removal ablations reported in the main paper, we further evaluate three variants that use each penalty group of the aesthetic cost in isolation. As shown in Tab. 5, in all three cases, performance is consistently worse than the LeLA-NoRefine baseline. Using only hard constraints (LeLA-Hard) increases AAC from 5.3e4 to 9.7e4 and also leads to more crossings and obstructions (AI 0.037 vs. 0.027, OR 6.32% vs. 2.87%). Using only major-violation penalties (LeLA-Major) further degrades arrangement quality, yielding the highest AI (0.047) and OR (6.90%) among the variants. The minor-only setting (LeLA-Minor) gives the worst AAC by a large margin (4.4e5) and a substantially higher obstruction rate (5.75%), even though it is the fastest. Overall, none of the single-group penalties matches the baseline, indicating that the three groups play complementary roles in shaping satisfactory arrangements.

Number of Candidate Paths. To evaluate how the number of candidate paths affects leader-line arrangement quality, we train LeLA for different values of K , where $K \in \{5, 10, 15, 20, 25, 30\}$. Figure 11 plots the performance (AAC and AI) of LeLA under these settings. Both metrics exhibit a non-monotonic dependence on K . When K is small (5 or 10), AAC and AI are relatively high, suggesting that limited candidate diversity restricts the model’s ability to find good arrangements. Increasing K to 20 yields the lowest AAC and AI, indicating the best trade-off between performance and candidate diversity. However, further increasing K beyond 20 again degrades both metrics, likely because the enlarged candidate set introduces more suboptimal paths and makes the selection problem harder. Overall, a moderate number of candidates ($K = 20$ in our setting) provides the most effective balance between exploration and arrangement quality.

Qualitative Results Figure 12 presents additional visualization comparisons on samples from the SWU-AMIL test set. These cases cover scenarios with high leader-line density and complex spatial constraints. The visualization compares LeLA’s output against baseline methods and the

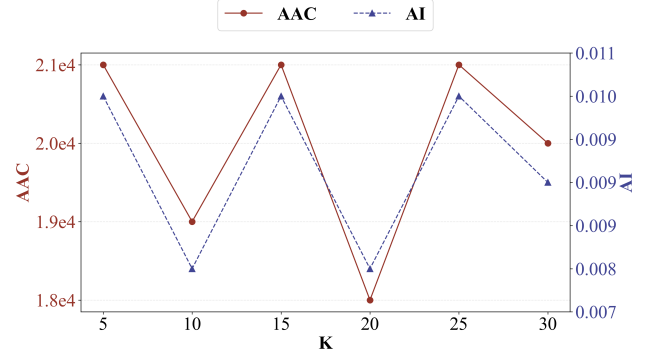


Figure 11. Ablation studies of the number of candidate paths. AAC (left axis) and AI (right axis) are plotted.

ground truth, focusing on adherence to aesthetic rules such as occlusion avoidance and intersection-free arrangements. In the first five examples, LeLA produces arrangements with fewer crossings, fewer occlusions of important content, and more regular geometry than the baselines. In the sixth and seventh examples, all methods struggle to fully satisfy the aesthetic constraints; these cases highlight particularly challenging configurations for current algorithms. In the last two examples, all methods achieve reasonably good layouts, and the gap between LeLA and the baselines narrows, suggesting that simpler instances can be handled adequately even by traditional approaches.

To further evaluate the generalization ability of LeLA, we use label layouts predicted by the state-of-the-art label placement method LPCE [43] as inputs to generate leader-line arrangements. The layouts produced by LPCE introduce challenges not present in the ground-truth data, such as imperfect label alignment and overlaps. As shown in Fig. 13, LeLA successfully generates plausible leader-line arrangements on these predicted layouts, demonstrating robustness to variations in label layout quality.

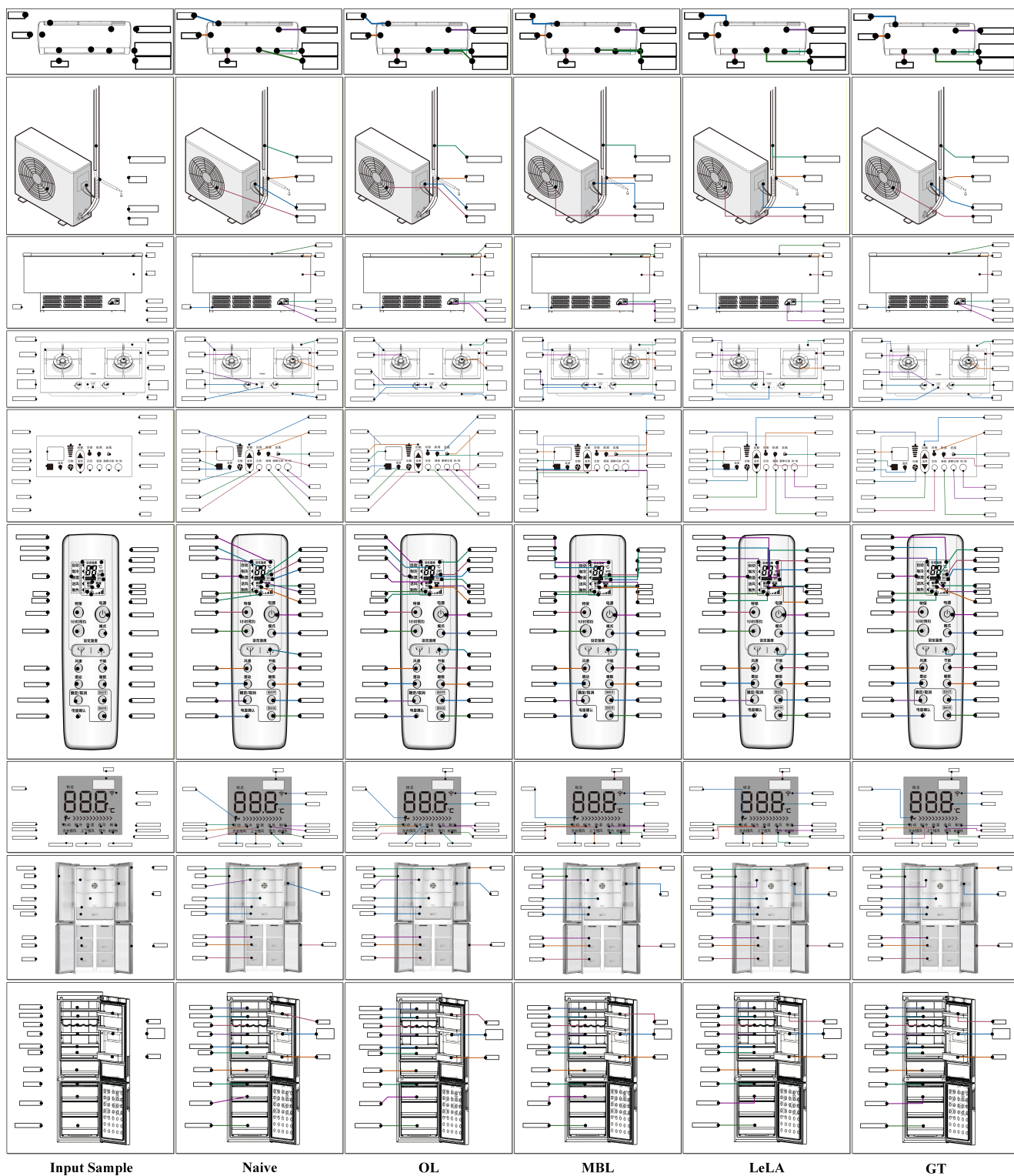


Figure 12. Visualization of results from different methods on the SWU-AMIL dataset.

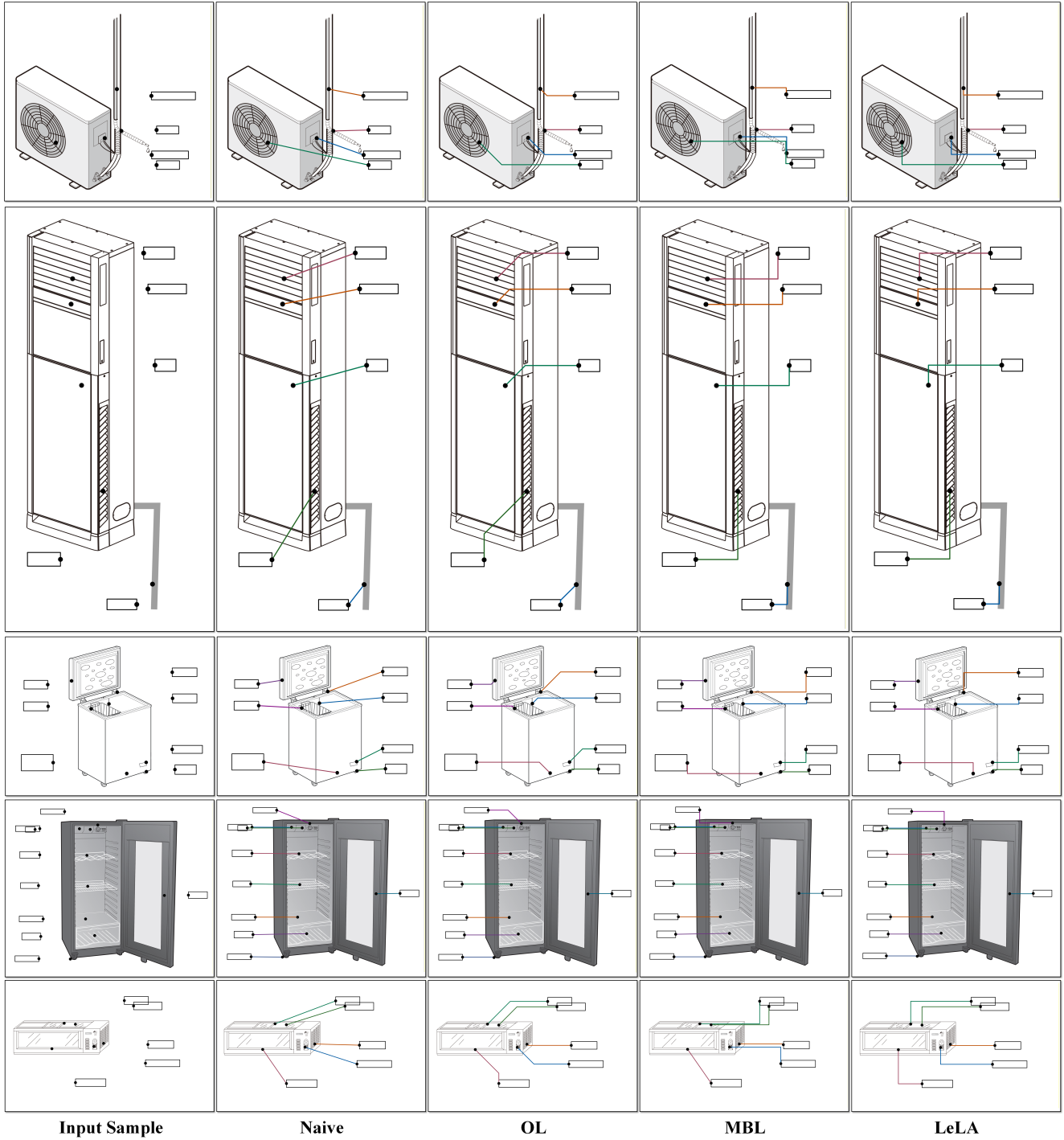


Figure 13. Visualization of results on label layouts predicted by LPCE [43].